

API Versioning Overview

Jaime Jimenez - jaime.jimenez@ericsson.com

February 5, 2024

Contents

1	API Versioning	2
2	Versioning Strategies	2
2.1	Additive-Change Strategy	2
2.2	Explicit-Version Strategy	2
3	Versioning Techniques	3
4	Version Semantics	3
5	Validation	4
5.1	API Version Validation	4
6	Communication Plan	5
7	Best API Design Practices	5
7.1	Keep Compatible Changes Out of Names	5
7.2	Avoid New Major Versions	5
7.3	Make Changes Backwards-Compatible	6
7.4	Think about Forwards-Compatibility	6
7.5	Version at Appropriate Granularities	6
7.6	Make Minor and Patch Information Discoverable	7
8	References	7

1 API Versioning

This document presents my own overview of the current state of the art in API versioning, focusing on best practices, strategies, techniques, and validation methods. The content is tailored to provide valuable insights for the OMA DMSO group, aiming to facilitate informed decision-making and the adoption of effective versioning approaches. The document is structured as follows:

1. **Versioning Strategies:** Discusses additive-change and explicit-version strategies.
2. **Versioning Techniques:** Explores common ways APIs handle versioning, such as URI versioning, query parameter versioning, custom request headers, content negotiation, and hostname versioning.
3. **Version Semantics:** Provides an overview of semantic versioning and its adoption in the industry.
4. **Validation:** Covers API version validation using regular expressions.
5. **Communication Plan:** Presents possible communication channels and timelines for backward-compatible and backward-incompatible changes in APIs.
6. **Best API Design Practices:** Offers a set of best practices for designing APIs that can evolve gracefully over time, with examples for each practice.

2 Versioning Strategies

Versioning is essential for bundling changes into understandable chunks and providing developers a way to understand your API. According to [1] there are two possible strategies for handling versioning: additive-change strategy and explicit-version strategy. The most common on the web is the latter. The additive-change strategy is less common and is often used in internal APIs or when the API is not exposed to the public.

2.1 Additive-Change Strategy

In an additive-change strategy, all updates are compatible with previous versions. This strategy requires avoiding backward-incompatible changes, such as:

- Removing or renaming APIs or API parameters
- Changing a type for a response field
- Changes in behavior for an existing API
- Changes in error codes and fault contracts

In this strategy, you make changes like adding an output field or a new API endpoint without changing a response field's type or removing a response field without letting users opt-in via request parameters. This means you'll add more and more parameters that will alter the response of an API request.

2.2 Explicit-Version Strategy

When creating an explicitly numbered versioning system, the first thing to decide is how users will interact with versions, what is called a versioning *scheme*. There are several choices available, each with distinct advantages. The version access pattern should be as stable as promised in accompanying documentation, and developers should have the option to opt into new versions while maintaining stability on previous versions.

Some strategies for defining version schemes include:

- Updating URI components
- Using HTTP headers
- Using request parameters

In addition to implementing the versioning system, you need to think about how to organize and label your versions. For this, the semantic versioning specification (SemVer) is the most commonly used (see [3]).

3 Versioning Techniques

There are several common ways in which APIs handle versioning [1]:

1. URI versioning: This involves including the version number directly in the API endpoint's URL. For example, `https://api.example.com/v1/users` or `https://api.example.com/v2/users`. This approach is simple and easy to understand but can lead to URL clutter.
2. Query parameter versioning: In this method, the version number is passed as a query parameter in the URL. For example, `https://api.example.com/users?version=1` or `https://api.example.com/users?version=2`. This method is also easy to implement, but it can result in long URLs with multiple query parameters.
3. Custom request header: The version information is included in a custom header field in the HTTP request. For example, `X-API-Version: 1` or `X-API-Version: 2`. This approach keeps the URL clean and allows for more flexibility, but it might be less visible and harder to debug.
4. Content negotiation (Accept header): This method uses the standard HTTP `Accept` header to specify the desired API version. The client requests a specific version by including it in the `Accept` header, like `Accept: application/vnd.example.api+json; version=1.0` or `Accept: application/vnd.example.api+json; version=2.0`. This approach follows the HTTP standard but may be more complex to implement and understand.
5. Hostname versioning: In this approach, different versions of the API are served from different subdomains. For example, `https://v1.api.example.com/users` or `https://v2.api.example.com/users`. This method is easy to understand and allows for clear separation of API versions, but it may require additional infrastructure setup.

4 Version Semantics

Semantic versioning is a widely adopted method for expressing versions in a standardized and meaningful way. In [5] of a collection of 7,114 APIs, they discovered that 5,292 APIs employed semantic versioning (with 112,908 commits) at some point in their history. They also state:

semantic versioning (SemVer) is the most widely adopted versioning scheme among API releases, specifically in the case of APIs that use the `info.version` field (60.56%). In contrast, for APIs that use alternative methods for versioning, such as embedding version identifiers in endpoints URLs or the server DNS name, the most common practice is to use shorter version identifiers that include only the major version counter. Additionally, for preview releases, SemVer is often combined with other tags to reflect the type of preview release, such as `Develop`, `Snapshot`, `Alpha`, `Beta`, or `Preview`

Therefore, it helps developers and users understand the changes between different versions of a software component, such as an API, library, or application. Semantic versioning uses a three-part version number in the format of `MAJOR.MINOR.PATCH`, where:

1. MAJOR: This number is incremented when there are backward-incompatible changes in the software, such as changes in the API that would break existing clients.
2. MINOR: This number is incremented when new functionality is added in a backward-compatible manner. Existing clients should continue to work without any issues, but they may not be able to access the new features.
3. PATCH: This number is incremented when backward-compatible bug fixes are introduced. These fixes should not affect the software's functionality or introduce new features.

For example, a version number like `2.3.4` denotes the second major version, with three minor updates and four patches.

In addition to the standard semantic versioning, some projects use additional techniques or labels to express more information about the version:

- Pre-release versioning: To indicate that a version is not yet stable or production-ready, pre-release labels can be added to the version number, e.g., `1.0.0-alpha`, `1.0.0-beta`, or `1.0.0-rc1` (release candidate 1).
- Build metadata: Additional information about the build or environment can be appended to the version number, separated by a `+` symbol, e.g., `1.0.0+build123`. This metadata does not affect the version precedence or compatibility.
- Date-based versioning: Some projects use a date-based versioning scheme, such as `YYYY.MM.DD` or `YYYY.MM`, to indicate when the version was released. While this is not strictly semantic versioning, it can provide useful information about the age and recency of a release.
- Hybrid versioning: Some projects combine semantic versioning with date-based or other versioning schemes, e.g., `2.0.0-20220202` or `2.0.0-rc1+build123`.

When using semantic versioning or any other versioning technique, it's essential to communicate the versioning scheme clearly in the project documentation and adhere to the chosen scheme consistently [1].

5 Validation

When working with APIs, it's essential to validate the version numbers to ensure compatibility and proper functionality. As mentioned before, numbers typically follow the Semantic Versioning (SemVer) format, which consists of three components: MAJOR.MINOR.PATCH (e.g., `1.2.3`).

5.1 API Version Validation

API version validation can be performed using a regular expression (regex) to ensure that the version number follows the SemVer format. The following regex (see [2]) can be used to validate a version number:

```
/^(?P<major>0|[1-9]\d*)\.(?P<minor>0|[1-9]\d*)\.(?P<patch>0|[1-9]\d*)
(?:-(?P<prerelease>(?:0|[1-9]\d*|\d*[a-zA-Z-][0-9a-zA-Z-]*)
(?:\.(?:0|[1-9]\d*|\d*[a-zA-Z-][0-9a-zA-Z-]*)*)?)?
(?:\+(?P<buildmetadata>[0-9a-zA-Z-]+(?:\.[0-9a-zA-Z-]+)*)?)?$/gm
```

This regex validates the version string and captures the major, minor, and patch versions, as well as optional prerelease and build metadata identifiers.

You can test and analyze this regex using the following URL: <https://regex101.com/r/Ly701x/3/>.

For example: suppose you have an API that follows the SemVer standard, and you want to validate the version string provided by a user. You can use the regex above to ensure the version string is valid and extract the components for further processing.

For example, given the version string `1.2.3-alpha.1+build.123`, the regex will validate the string and capture the following components:

- Major: `1`
- Minor: `2`
- Patch: `3`
- Prerelease: `alpha.1`
- Build metadata: `build.123`

By validating and capturing the version components, you can ensure your API operates correctly and maintains compatibility with various clients and integrations.

6 Communication Plan

Communication plans for API changes are done to ensure that developers have a mechanism to receive updates (see [1] page 134). Starting with a RSS feed is a good approach, but eventually, you'll want a way to communicate with specific developers about changes that affect them. For example, if you previously used `repositories.fetch` and introduced `repositories.fetchSingle`, you might want to contact all developers who used the `repositories.fetch` endpoint in the past 12 months to inform them about the new `repositories.fetchSingle` endpoint. You might also want to contact developers who used the `repositories.fetch` endpoint and received a 500-level response after releasing a product that caused timeouts.

The table below shows possible communication channels and timelines for backward-compatible and backward-incompatible changes. In this example, backward-compatible changes can be released at any time, and developers will be given 18 months' notice before backward-incompatible changes are released.

Change Type	Examples	Communication Channels	Time to Release After Notification
Backward compatible	Request parameter added, response field added, new API method added	RSS feed, API docs	Anytime
Backward incompatible	Response field removed, functionality changes, response type changes, endpoint removed	RSS feed, API docs, Email to affected developers, Blog post explaining change	18 months

In addition to communication mechanisms where you broadcast information to specific audiences, there are other built-in ways to communicate. You might consider annotating response payloads or headers with information about changes.

7 Best API Design Practices

This section is mostly based on [4] insights into the best practices for designing APIs that can evolve gracefully over time, along with examples for each practice:

7.1 Keep Compatible Changes Out of Names

Names should remain stable over time and should identify a known set of semantics. This applies to all identifiers, including URLs, media types, link relation names, and HTTP headers. Only include the major version number in the URL, leaving out minor and patch versions, as they don't require name changes for backwards compatibility.

Example:

Instead of using URLs like `http://api.example.com/v1.2.3/ladder`, only include the major version number:

`http://api.example.com/v1/ladder`

Or even use a different hostname for each major version:

`http://api2.example.com/ladder`

7.2 Avoid New Major Versions

Introducing new major versions requires significant effort from both API developers and users. To minimize the need for new major versions, focus on making changes backwards-compatible and ensuring that your API can evolve without breaking existing clients.

Example:

Suppose you have an API for managing a list of products:

```
http://api.example.com/v1/products
```

Instead of introducing a new major version for every small change, focus on making updates that are compatible with the existing version, minimizing the need for major version updates.

7.3 Make Changes Backwards-Compatible

Strive to make your API changes backwards-compatible by adding new resources, methods, or formats without affecting existing functionality. Even removing support for a feature can be considered backwards-compatible if it only affects clients that use it.

Example:

If you want to add a new field to the product resource, simply add the field without changing the existing structure:

```
{
  "id": 1,
  "name": "Product A",
  "price": 100,
  "new_field": "New Value"
}
```

Clients that don't expect the new field can continue to work without issues.

7.4 Think about Forwards-Compatibility

Consider how your clients will handle future changes and set clear expectations for their behavior. Encourage clients to be flexible and handle unexpected input gracefully. Document your expectations to guide developers in building resilient clients.

Example:

In your API documentation, specify that clients should ignore any unknown fields in JSON responses:

```
{
  "id": 1,
  "name": "Product A",
  "price": 100,
  "unknown_field": "Value"
}
```

By doing this, you allow your API to introduce new fields without breaking existing clients.

7.5 Version at Appropriate Granularities

Ensure that your versioning strategy reflects the appropriate level of granularity. For instance, a version number in a URL should represent a collection of resources and functionality, not just a single resource. Keep format changes separate from API versions to maximize their value and flexibility.

Example:

A version number in a URL represents a set of resources and functionality:

```
http://api.example.com/v1/products
http://api.example.com/v1/users
```

Format changes should be separate from API versions:

```
http://api.example.com/v1/products (application/json)
http://api.example.com/v1/products (application/xml)
```

7.6 Make Minor and Patch Information Discoverable

Although minor and patch version numbers should not be included in names, they still hold value in other contexts, such as release notes and documentation. Make this information discoverable by clients through direct interrogation of the interface or by including it in the Server or User-Agent headers.

Example:

Include the minor and patch version information in the Server header:

Server: Example-API/1.2.3

By following these best practices, you can design APIs that are more maintainable, flexible, and capable of evolving without causing unnecessary disruptions for your users.

8 References

- [1] *Designing Web APIs [Book]*. URL: <https://www.oreilly.com/library/view/designing-web-apis/9781492026914/> (visited on 02/02/2024).
- [2] Firas Dib. *Regex101: Build, Test, and Debug Regex*. regex101. URL: <https://regex101.com/> (visited on 02/02/2024).
- [3] Souhaila Serbout and Cesare Pautasso. “An Empirical Study of Web API Versioning Practices”. In: *Web Engineering*. Ed. by Irene Garrigós, Juan Manuel Murillo Rodríguez, and Manuel Wimmer. Vol. 13893. Cham: Springer Nature Switzerland, 2023, pp. 303–318. ISBN: 978-3-031-34443-5 978-3-031-34444-2. DOI: 10.1007/978-3-031-34444-2_22. URL: https://link.springer.com/10.1007/978-3-031-34444-2_22 (visited on 02/02/2024).
- [4] *Web API Versioning Smackdown*. Mark Nottingham. URL: https://www.mnot.net/blog/2011/10/25/web_api_versioning_smackdown (visited on 02/02/2024).
- [5] Laerte Xavier et al. “Historical and Impact Analysis of API Breaking Changes: A Large-Scale Study”. In: *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER). Feb. 2017, pp. 138–147. DOI: 10.1109/SANER.2017.7884616. URL: <https://ieeexplore.ieee.org/abstract/document/7884616> (visited on 02/02/2024).